AGENT RELIABILITY LAB

Controlled Fault Injection and Evidence-Based Failure Analysis for AI Agents

A Report Submitted in
Partial Fulfilment for award
of Bachelor of Technology

In

COMPUTER SCIENCE & ENGINEERING

CAPSTONE PROJECT 1

BCSE0758

By

SURYANSH SINGH (2301330100208)
SIMRAN KUMARI (2301330100200)
TANISHKA SAHU (2301330100210)

Under the Supervision of

Mr. Pradeep Kumar
Assistant Professor



Department of Computer Science & Engineering
School of Computer Science and Information Technology
NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY
GREATER NOIDA
(An Autonomous Institute)
Affiliated to
DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW

Session 2026-27

DECLARATION

We hereby declare that the work presented in this report entitled **Agent Reliability Lab: Controlled Fault Injection and Evidence-Based Failure Analysis for AI Agents** was carried out by us. We have not submitted the matter embodied in this report for the award of any degree or diploma from any other University or Institute.

Name	Roll Number	Signature
Suryansh Singh	2301330100208	
Simran Kumari	2301330100200	
Tanishka Sahu	2301330100210	

(Candidates' Signatures)

CERTIFICATE

Certified that **Suryansh Singh (2301330100208)**, **Simran Kumari (2301330100200)** and **Tanishka Sahu (2301330100210)** have carried out the project work presented in this Project Report entitled **Agent Reliability Lab** in partial fulfillment of the requirements for the award of **Bachelor of Technology, Computer Science and Engineering, 4th Year 7th Semester** (Capstone Project 1, BCSE0758) from Noida Institute of Engineering & Technology under my supervision.

Signature
Mr. Pradeep Kumar
Assistant Professor
NIET, Gr. Noida

Signature
Prof. Tripti Sharma
HOD
Department of CSE
NIET, Gr. Noida

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to **Mr. Pradeep Kumar**, Assistant Professor, for his invaluable guidance, support, and constant supervision throughout this project. His questions about how agents fail, and how a claim should be measured, shaped the direction of the work.

We extend our thanks to our esteemed HOD, **Prof. Tripti Sharma**, for her continuous motivation and for providing the academic environment and resources that made this project possible.

Finally, we are thankful to our peers, the project coordination committee, and everyone who directly or indirectly contributed to this work with their knowledge and feedback.

ABSTRACT

AI agents built on large language models plan, call tools, read observations, and retry after errors, yet they are usually evaluated with a single success rate measured once per task in an environment where nothing breaks. That number hides why runs fail, how consistently an agent succeeds, and what its reliability costs. This report presents *Agent Reliability Lab*, an open-source Python laboratory for controlled reliability experiments on any agent that maps a task to a recorded trajectory.

The laboratory injects tool timeouts and outages on a seeded schedule that depends only on the task, trial, tool, and call number, so every agent variant meets exactly the same failures and their runs form matched pairs. Each task is attempted several times, and the system reports success with confidence intervals, pass@k, pass^k, consistency, cost, latency, recovery, and step use. A rule-based classifier explains every failed run as a planning, tool-misuse, context, recovery, or horizon failure with evidence, and leaves unexplained failures unclassified. An exact McNemar test decides whether two variants differ, and a configuration search returns the Pareto front over success, cost, and latency together with the cheapest setting that meets a reliability target. Results are stored in SQLite and rendered as HTML reports through a command-line interface.

The implementation has 17 modules and 81 automated tests. Experiments with offline reference agents on a twelve-task customer-support suite show that the measured effect of injected faults matches its closed-form expectation, that the classifier recovers every planted defect with a precision of 1.00, and that under 30% tool timeouts two retries per call is the cheapest setting with at least 95% success. Future work evaluates live LLM agents through the included OpenAI-compatible adapter.

TABLE OF CONTENTS

Page No.

Declaration..... i
Certificate..... ii
Acknowledgement..... iii
Abstract..... iv
CHAPTER 1: INTRODUCTION..... 1
1.1 Overview..... 1
1.2 Problem Statement..... 1
1.3 Objective and Scope..... 1
1.4 Organization of the Report..... 2
CHAPTER 2: LITERATURE REVIEW..... 3
2.1 Overview of Existing Systems..... 3
2.2 Limitations of Existing Systems..... 4
2.3 How This Project Addresses Them..... 4
CHAPTER 3: REQUIREMENTS..... 5
3.1 Functional Requirements..... 5
3.2 Non-Functional Requirements..... 5
3.3 Software and Hardware Requirements..... 6
CHAPTER 4: SYSTEM DESIGN AND IMPLEMENTATION..... 7
4.1 System Architecture..... 7
4.2 Interface Design..... 7
4.3 Database Design..... 10
4.4 Design Diagrams..... 11
4.5 Implementation Details..... 17
CHAPTER 5: TESTING..... 22
5.1 Testing Methodology..... 22
5.2 Test Cases and Results..... 22
5.3 Experimental Evaluation..... 23
CHAPTER 6: CONCLUSION AND FUTURE WORK..... 27
6.1 Conclusion..... 27
6.2 Future Work..... 27
APPENDIX..... 28
REFERENCES..... 29

LIST OF FIGURES

Page No.

Figure 4.1: System architecture..... 7
Figure 4.2: Terminal output of relai run..... 8
Figure 4.3: HTML report: ranking, paired test, failure modes..... 9
Figure 4.4: HTML report: steps and evidence of two runs..... 10
Figure 4.5: Entity-relationship diagram of the run store..... 10
Figure 4.6: Use case diagram..... 11
Figure 4.7: Class diagram..... 12
Figure 4.8: Sequence diagram of relai run..... 13
Figure 4.9: Level 0 data flow diagram..... 13
Figure 4.10: Level 1 data flow diagram..... 14
Figure 4.11: Flowchart of the evaluation harness..... 15
Figure 4.12: Flowchart of failure classification..... 16
Figure 4.13: Flowchart of configuration search..... 17
Figure 4.14: Fault decision for one tool call..... 18
Figure 4.15: Evaluation loop with trials and crash capture..... 18
Figure 4.16: Recovery episodes..... 19
Figure 4.17: Pareto front..... 20
Figure 4.18: A three-call task in the reliability suite..... 20
Figure 4.19: Retry policy of the reference agent..... 20
Figure 5.1: Test and lint output..... 23
Figure 5.2: Success (solid) and pass^5 (dashed) as timeouts rise..... 24
Figure 5.3: Measured success against the closed form..... 25

LIST OF TABLES

Page No.

Table 2.1: Survey of existing systems....3
Table 2.2: Limitations and the approach taken....4
Table 3.1: Functional requirements....5
Table 3.2: Non-functional requirements....5
Table 3.3: Software and hardware requirements....6
Table 4.1: Command-line interface....8
Table 4.2: Database schema....11
Table 5.1: Representative test cases....22
Table 5.2: Distribution of automated tests....23
Table 5.3: Agents under 30% tool timeouts....24
Table 5.4: Measured success against the closed-form expectation....24
Table 5.5: Diagnosed root causes per planted defect (60 runs each)....25
Table 5.6: Sweep of the retry budget at 30% timeouts....25

CHAPTER 1
INTRODUCTION

Large language models (LLMs) are now used as agents: programs that break a goal into steps, call external tools such as search, databases, and calculators, read the results, and continue until they can answer. Customer support, coding, data analysis, and scheduling assistants are built this way. Before such an agent is trusted with real work, its developers need to know three things: why it fails when it fails, whether it succeeds every time or only sometimes, and how much its reliability costs. Agent Reliability Lab is a software laboratory that answers these questions with controlled, repeatable experiments.

1.1 Overview

The laboratory is distributed as a Python package named `relai` with a command-line interface. A researcher selects or writes a task suite, plugs in an agent through a two-member interface (a name and a `run(task)` method that returns a trajectory), and runs an experiment. During the experiment every tool call passes through a tool environment that injects timeouts and outages on a seeded schedule. Every task is attempted several times. The resulting trajectories are scored, measured, and diagnosed, and the variants are ranked, compared with a significance test, and searched for the cheapest reliable configuration.

The project works offline. It ships reference agents whose behaviour is known in advance, which allows the instrument itself to be validated without any model API, and it includes an adapter for live LLMs served through any OpenAI-compatible endpoint, including local models through Ollama.

1.2 Problem Statement

Existing agent benchmarks report task success measured once per task, and existing tracing tools record logs of production traffic. Neither lets a team run a fair experiment in which two agents meet the same tool failures, every failure is explained with evidence, the consistency of success across attempts is measured, and the cost of reliability is visible. The problem addressed by this project is to design and implement such a laboratory for any agent, with metrics and analysis computed only from recorded trajectories so that every number is reproducible.

1.3 Objective and Scope

The objectives of the project are:

1. **Evaluate agents** on success, correctness, cost, latency, token use, tool calls, recovery, and step use, and on reliability across repeated trials (pass@k, pass^k, consistency).
2. **Explain failures and recovery** with seeded fault injection and an evidence-based taxonomy of five failure modes, validated against agents with planted defects.
3. **Enable controlled experiments and optimization** with identical fault schedules for all variants, confidence intervals, a paired significance test, and a cost-aware configuration search.

The scope of the current version includes:

- A typed trajectory model and a minimal agent interface admitting any agent implementation.
- A tool environment with mock tools and a seeded fault policy (timeouts, outages, fail-first-N).
- A harness with repeated trials, timing, and crash capture; scorers; metric derivation; and failure classification.
- Statistics (Wilson and bootstrap intervals, exact McNemar test) and optimization (grid sweep, Pareto front, recommendation).
- YAML task suites, reference agents, a classifier self-check, SQLite and JSON Lines storage, HTML and Markdown reports, and a command-line interface.
- An adapter for live LLM agents through OpenAI-compatible APIs.

A hosted monitoring service, a graphical dashboard, and model training are outside the scope of this phase and are listed as future work.

1.4 Organization of the Report

Chapter 2 reviews existing systems and their limitations. Chapter 3 states the requirements. Chapter 4 describes the architecture, interface, database, design diagrams, and implementation. Chapter 5 covers testing and experimental results. Chapter 6 concludes and lists future work. The appendix gives the source code location, commands, and the division of work.

CHAPTER 2
LITERATURE REVIEW

2.1 Overview of Existing Systems

Agent benchmarks. AgentBench, GAIA, SWE-bench, and WebArena provide tasks and environments for coding, web navigation, and general assistance, and report task success. tau-bench introduced simulated tool-agent-user conversations and the pass^k metric, the chance that all of k attempts succeed; with a strong model, pass^8 on its retail domain falls below 25% even though single-attempt success exceeds 60%.

Cost-aware and reliability-focused evaluation. Kapoor et al. showed that agent evaluation must control for cost and that simple baselines can match complex agents on an accuracy and cost Pareto curve. The Holistic Agent Leaderboard (HAL) standardizes cost-aware evaluation at scale with complete trace logging. Rabanser et al. (ICML 2026) proposed twelve reliability metrics across consistency, robustness, predictability, and safety, and found that recent capability gains brought only small reliability gains.

Fault injection. Chaos engineering deliberately injects failures into production systems to build confidence in their resilience. For agents, ToolEmu emulates tools with an LLM, ToolMisuseBench offers deterministic, replayable tool faults under budgets, ToolBench-X injects recoverable tool-environment hazards, and AgentChaos injects faults into LLM API responses through a proxy.

Failure taxonomies and attribution. MAST defines fourteen failure modes for multi-agent systems from annotated traces, TRAIL annotates errors in long agent traces, and Who&When studies automated attribution of a failure to an agent and step; its best method identifies the failing step in only 14.2% of cases.

Evaluation frameworks and platforms. HELM and the LM Evaluation Harness benchmark models. Inspect AI is a research-grade evaluation framework with sandboxes. DeepEval and Promptfoo run tests in continuous integration. LangSmith, Langfuse, and Arize Phoenix trace production traffic and score it. Table 2.1 summarizes these approaches.

Table 2.1: Survey of existing systems

System / work	Focus	Strength	Gap for this problem
AgentBench, GAIA, SWE-bench, WebArena	Agent task suites	Realistic tasks	Success once per task; no diagnosis
tau-bench	Tool-agent-user tasks	pass^k reliability metric	No injected faults
HAL	Cost-aware leaderboard	Scale, cost tracking, logs	No shared fault schedule
Rabanser et al. 2026	Reliability metrics	Twelve-metric profile	No paired test or classifier check
ToolMisuseBench, ToolBench-X	Tool fault benchmarks	Replayable faults, hazards	Datasets, not a paired factor
AgentChaos	API fault proxy	Non-intrusive injection	No agent failure taxonomy

System / work	Focus	Strength	Gap for this problem
MAST, TRAIL, Who&When	Failure taxonomies	Rich human-annotated labels	Labels not checked against ground truth
LangSmith, Langfuse, Phoenix	Production tracing	Observability at scale	Observe traffic; no experiments
Inspect AI, DeepEval, Promptfoo	Evaluation frameworks	Breadth, CI integration	No recovery metric or fault schedule

2.2 Limitations of Existing Systems

Across these systems, five limitations recur. First, **success-only reporting**: a single success rate does not say why a run failed or how consistent the agent is. Second, **unfair comparison**: when failures are random per run, two agents never meet the same conditions, so differences between them mix agent quality with luck. Third, **missing recovery accounting**: tool timeouts are common in practice, yet the quality of retries is rarely measured. Fourth, **unverified diagnosis**: automated failure labels, whether rule-based or produced by LLM judges, are seldom checked against runs whose true failure is known. Fifth, **cost-blind selection**: a more reliable setting is often chosen without reporting the extra money and time it requires.

2.3 How This Project Addresses Them

Table 2.2: Limitations and the approach taken

Limitation	Approach in Agent Reliability Lab
Success-only reporting	Repeated trials with pass@k, pass^k, consistency; five failure modes with evidence
Unfair comparison	Fault schedule depends only on seed, task, trial, tool, and call number; exact McNemar paired test
Missing recovery accounting	Recovery episodes per error; recovery rate; recovery failure mode
Unverified diagnosis	Self-check with reference agents carrying one planted defect each; rules pinned by tests
Cost-blind selection	Cost and latency per run; Pareto front; cheapest setting meeting a target

The project does not claim that fault injection or reliability metrics are new. Its contribution is the combination: agent-independent fault schedules that make runs paired, a fault injector validated against its analytic expectation, a classifier validated with planted defects, and cost-aware selection, in one small library that any agent can plug into.

CHAPTER 3
REQUIREMENTS

3.1 Functional Requirements

The system must support the following functions:

Table 3.1: Functional requirements

ID	Requirement
FR1	Load task suites from YAML or JSON with tasks, expected answers, permitted tools, tags, step budgets, mock tools, default faults, and reference tool plans; reject suites with undefined tools or duplicate ids.
FR2	Accept any agent exposing a name and run(task) that returns a trajectory: functions, scripted traces, reference agents, or LLMs behind an OpenAI-compatible API.
FR3	Inject reproducible tool faults (timeouts, outages, fail-first-N) whose schedule depends only on seed, task, trial, tool, and call number.
FR4	Run each task for k trials; report success, pass@k, pass^k, and consistency.
FR5	Score answers with exact, contains, normalized, numeric (tolerance and partial credit), and regular-expression scorers.
FR6	Derive cost, step and wall latency, tokens, tool calls, repeated calls, recovery rate, and step count from each trajectory.
FR7	Classify runs into planning, tool-misuse, context, recovery, and horizon failures with evidence and a root cause; mark unmatched failures unclassified.
FR8	Compare variants with confidence intervals and an exact paired McNemar test.
FR9	Sweep agent settings over a grid; return the Pareto front and the cheapest setting meeting a success target.
FR10	Store experiments in SQLite and JSON Lines, reload them without loss, and render HTML and Markdown reports.
FR11	Check the classifier against reference agents with planted defects.
FR12	Offer all functions through commands (demo, suites, run, sweep, stress, selfcheck, report) and a Python API.

3.2 Non-Functional Requirements

Table 3.2: Non-functional requirements

Quality	Requirement and how it is met
Reproducibility	The same seed gives the same faults and numbers; ties in ranking are broken deterministically.
Fairness	Every variant meets the identical fault schedule, so runs are paired.
Performance	About 6,500 scored and classified simulated runs per second; the full test suite runs in under one second.
Portability	Python 3.11 to 3.13 on Linux, macOS, or Windows; two runtime dependencies; no GPU or network for offline use.
Extensibility	New agents, scorers, suites, fault kinds, and failure modes attach without changing the harness.
Robustness	An exception inside an agent becomes a failed run, never a crashed experiment.
Security	No eval(): the calculator parses arithmetic with a whitelist; HTML reports escape all agent output; API keys come from the environment.
Maintainability	Typed Pydantic models, one test file per module, Ruff lint, CI on three Python

Quality	Requirement and how it is met
	versions.
Usability	One command per experiment; terminal tables with intervals; standalone HTML reports.

3.3 Software and Hardware Requirements

Table 3.3: Software and hardware requirements

Category	Requirement
Language	Python 3.11 or later
Runtime libraries	Pydantic 2.7 or later (typed models), PyYAML 6 or later (task suites)
Standard library	sqlite3 (store), urllib (LLM adapter), statistics, random, ast
Development tools	pytest, Ruff, Git, GitHub Actions
Optional services	OpenAI-compatible API (OpenAI, Groq, vLLM) or Ollama for local models
Editor	Visual Studio Code or any Python IDE
Operating system	Windows 10/11, macOS, or any Linux distribution
Processor	Dual-core x86-64 or ARM processor; no GPU needed
Memory and storage	4 GB RAM minimum (8 GB recommended); 500 MB free disk

CHAPTER 4
SYSTEM DESIGN AND IMPLEMENTATION

4.1 System Architecture

The system follows a layered, batch architecture (Fig. 4.1) with one strict rule: only the execution layer calls agents, and every later layer reads recorded trajectories.

1. **Input layer:** task suites (YAML or JSON) with mock tools and default faults, and the agent under test behind the name plus run(task) interface.
2. **Execution layer:** the tool environment decides each fault from the seed; the recorder turns tool calls into steps; the harness runs every variant on every task for k trials, times each run, and captures crashes.
3. **Analysis layer:** scorers, metric derivation, and the failure classifier read finished trajectories.
4. **Output layer:** reports rank variants; statistics add intervals and a paired test; the optimizer computes the Pareto front; the store saves runs; the reporter renders HTML and Markdown.

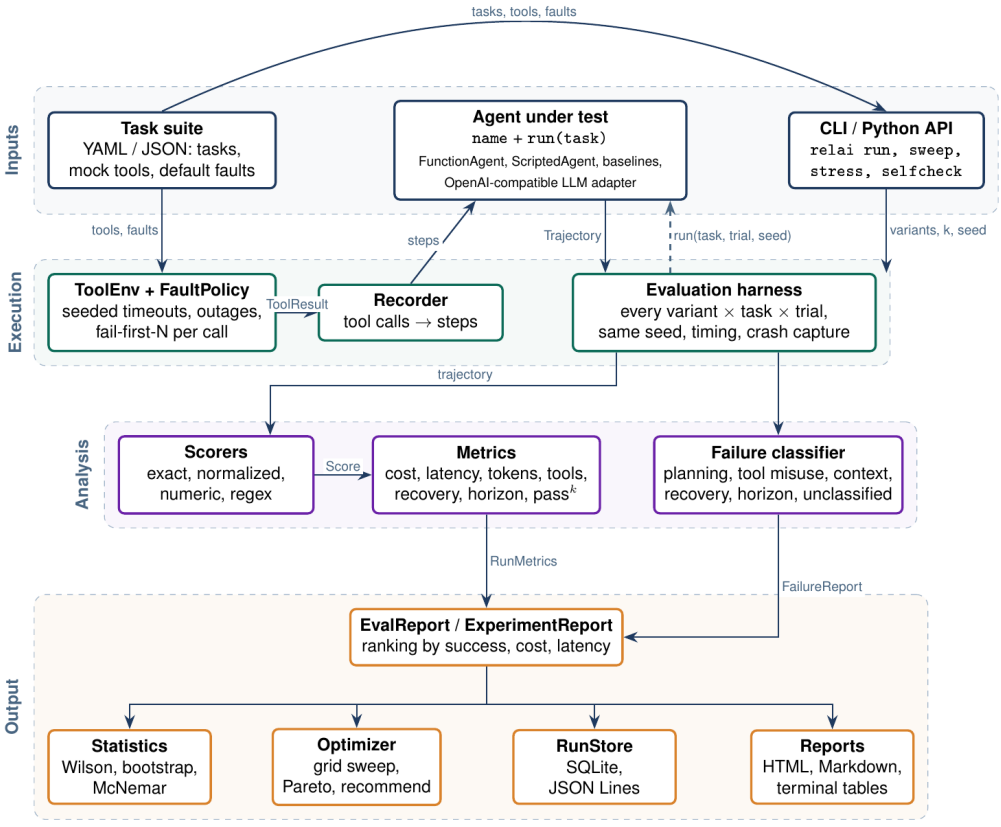


Figure 4.1: System architecture

4.2 Interface Design

The user interface is a command-line tool, `relai`, with a Python API behind it. Each command performs one experiment type (Table 4.1). Terminal output shows a ranked table with

confidence intervals and the paired test (Fig. 4.2); the HTML report shows the ranking, Pareto marks, failure-mode bars, and every run with its steps and evidence (Figs. 4.3 and 4.4).

Table 4.1: Command-line interface

Command	Purpose
relai demo	Compare a brittle and a resilient scripted agent
relai suites	List built-in task suites
relai run SUITE --agent ... --trials k --timeout-rate p	Evaluate agents under seeded faults; --html, --markdown, --json, --store
relai sweep SUITE --param max_retries=0,1,2	Search settings; print Pareto front and recommendation
relai stress SUITE --rates 0,0.1,...	Success as the timeout rate rises
relai selfcheck	Classifier check against planted defects
relai report runs.db --list | --html	List or render stored experiments

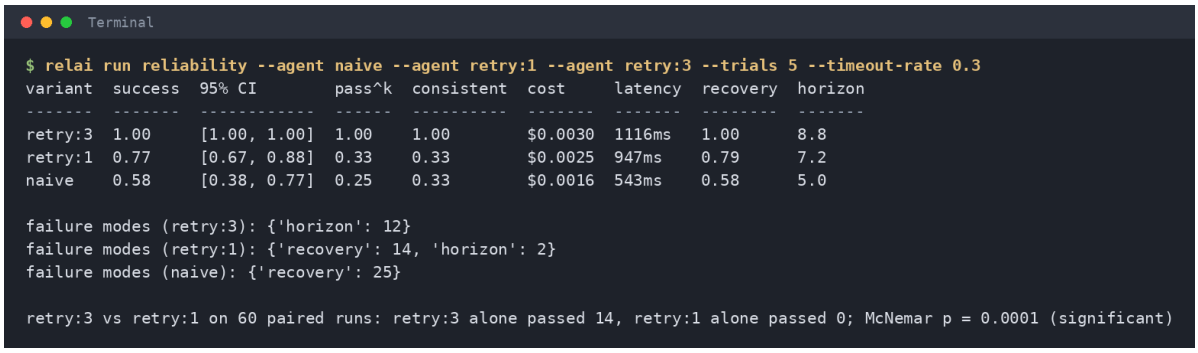


Figure 4.2: Terminal output of relai run

Agent Reliability Lab: naive vs retry

Suite: reliability. Variants: 3. Tasks: 12. Trials per task: 5. Seed: 0.

Ranking

#	variant	success [95% CI]	pass^k	consistency	cost / run	latency	wall	recovery	horizon	top root cause
1	**retry:3 ★**	1.00 [1.00, 1.00]	1.00	1.00	$0.0030	1116 ms	0.09 ms	1.00	8.8	horizon
2	**retry:1 ★**	0.77 [0.67, 0.88]	0.33	0.33	$0.0025	947 ms	0.08 ms	0.79	7.2	recovery
3	**naive ★**	0.58 [0.38, 0.77]	0.25	0.33	$0.0016	543 ms	0.06 ms	0.58	5.0	recovery

★ marks variants on the Pareto front: no other variant is at least as good on success, cost, and latency while better on one of them.

Paired comparison

retry:3 vs retry:1 on 60 paired runs: 14 won only by retry:3, 0 only by retry:1. Exact McNemar p = 0.0001, so a significant difference at the 0.05 level.

Failure modes

retry:3

planning	0
tool_misuse	0
context	0
recovery	0
horizon	12
unclassified	0

retry:1

planning	0
tool_misuse	0
context	0
recovery	14
horizon	2
unclassified	0

naive

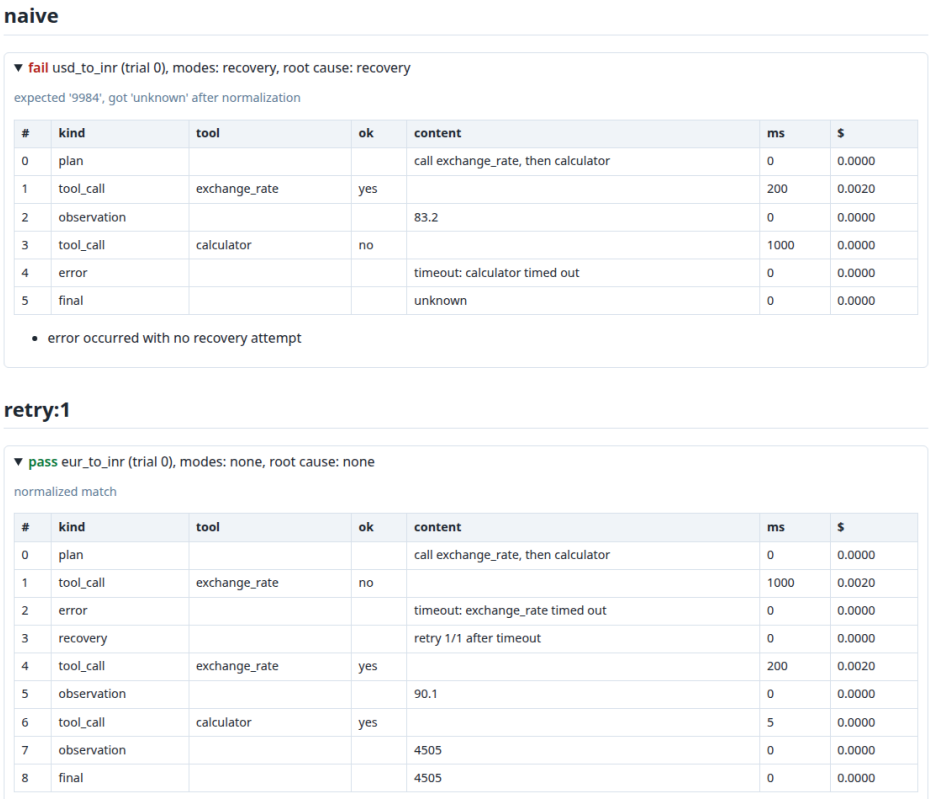
planning	0
tool_misuse	0
context	0
recovery	25
horizon	0
unclassified	0

Runs

retry:3

- ▸ pass order_status (trial 0), modes: none, root cause: none
- ▸ pass order_status (trial 1), modes: none, root cause: none
- ▸ pass order_status (trial 2), modes: none, root cause: none
- ▸ pass order_status (trial 3), modes: none, root cause: none
- ▸ pass order_status (trial 4), modes: none, root cause: none
- ▸ pass customer_id (trial 0), modes: none, root cause: none
- ▸ pass customer_id (trial 1), modes: none, root cause: none
- ▸ pass customer_id (trial 2), modes: none, root cause: none
- ▸ pass customer_id (trial 3), modes: none, root cause: none
- ▸ pass customer_id (trial 4), modes: none, root cause: none

Figure 4.3: HTML report: ranking, paired test, failure modes



naive

▼ **fail** usd_to_inr (trial 0), modes: recovery, root cause: recovery

expected '9984', got 'unknown' after normalization

#	kind	tool	ok	content	ms	$
0	plan			call exchange_rate, then calculator	0	0.0000
1	tool_call	exchange_rate	yes		200	0.0020
2	observation			83.2	0	0.0000
3	tool_call	calculator	no		1000	0.0000
4	error			timeout: calculator timed out	0	0.0000
5	final			unknown	0	0.0000

- error occurred with no recovery attempt

retry:1

▼ **pass** eur_to_inr (trial 0), modes: none, root cause: none

normalized match

#	kind	tool	ok	content	ms	$
0	plan			call exchange_rate, then calculator	0	0.0000
1	tool_call	exchange_rate	no		1000	0.0020
2	error			timeout: exchange_rate timed out	0	0.0000
3	recovery			retry 1/1 after timeout	0	0.0000
4	tool_call	exchange_rate	yes		200	0.0020
5	observation			90.1	0	0.0000
6	tool_call	calculator	yes		5	0.0000
7	observation			4505	0	0.0000
8	final			4505	0	0.0000

Figure 4.4: HTML report: steps and evidence of two runs

4.3 Database Design

Experiments are stored in a SQLite database with five tables (Fig. 4.5, Table 4.2). An experiment contains variants, a variant produces runs, and a run records steps and diagnosed failure modes. Every foreign key cascades on delete. Each run keeps its full record in `result_json`, so stored experiments reload without loss, while the `steps` and `failures` tables allow SQL queries such as failure counts per variant.

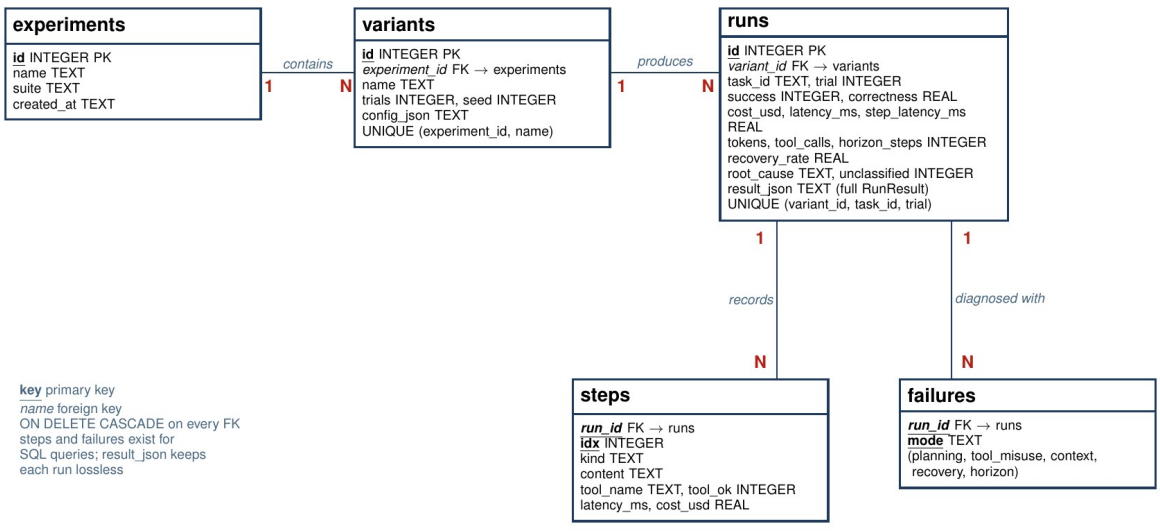


Figure 4.5: Entity-relationship diagram of the run store

Table 4.2: Database schema

Table	Key columns	Purpose
experiments	id (PK), name, suite, created_at	One saved experiment
variants	id (PK), experiment_id (FK), name, trials, seed, config_json	One agent configuration
runs	id (PK), variant_id (FK), task_id, trial, success, cost_usd, step_latency_ms, root_cause, result_json	One trial of one task
steps	run_id (FK), idx, kind, content, tool_name, tool_ok, latency_ms, cost_usd	Steps of a run
failures	run_id (FK), mode	Diagnosed failure modes

4.4 Design Diagrams

Use cases (Fig. 4.6). The researcher defines suites, configures faults, plugs in agents, runs evaluations (which include scoring and classification), compares variants, sweeps settings, stress-tests, runs the self-check, and stores and reports results.

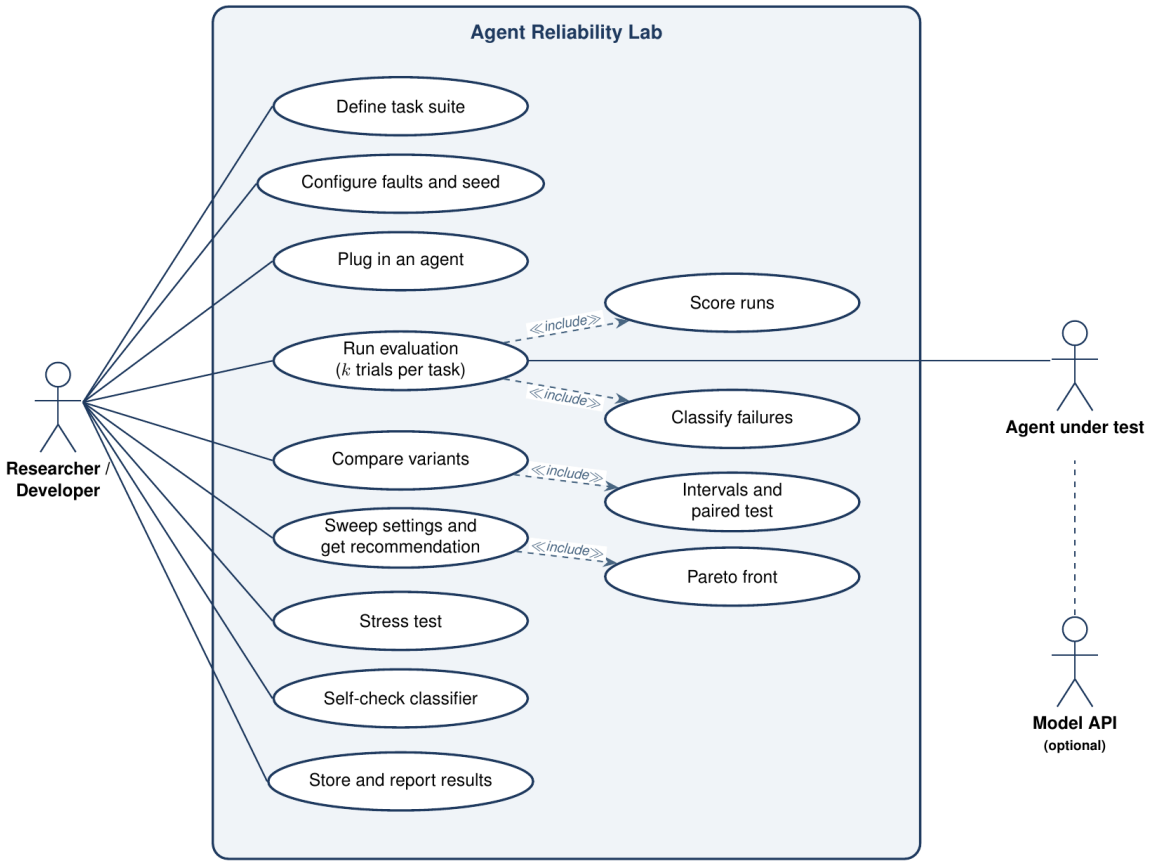


Figure 4.6: Use case diagram

Classes (Fig. 4.7). A RunResult composes a Trajectory with its Score, RunMetrics, and FailureReport; EvalReport groups the runs of one variant and ExperimentReport groups variants. Every agent implements the Agent protocol; the reference agents share PolicyAgent, and the looping and careless agents extend RetryAgent so that each carries exactly one defect.

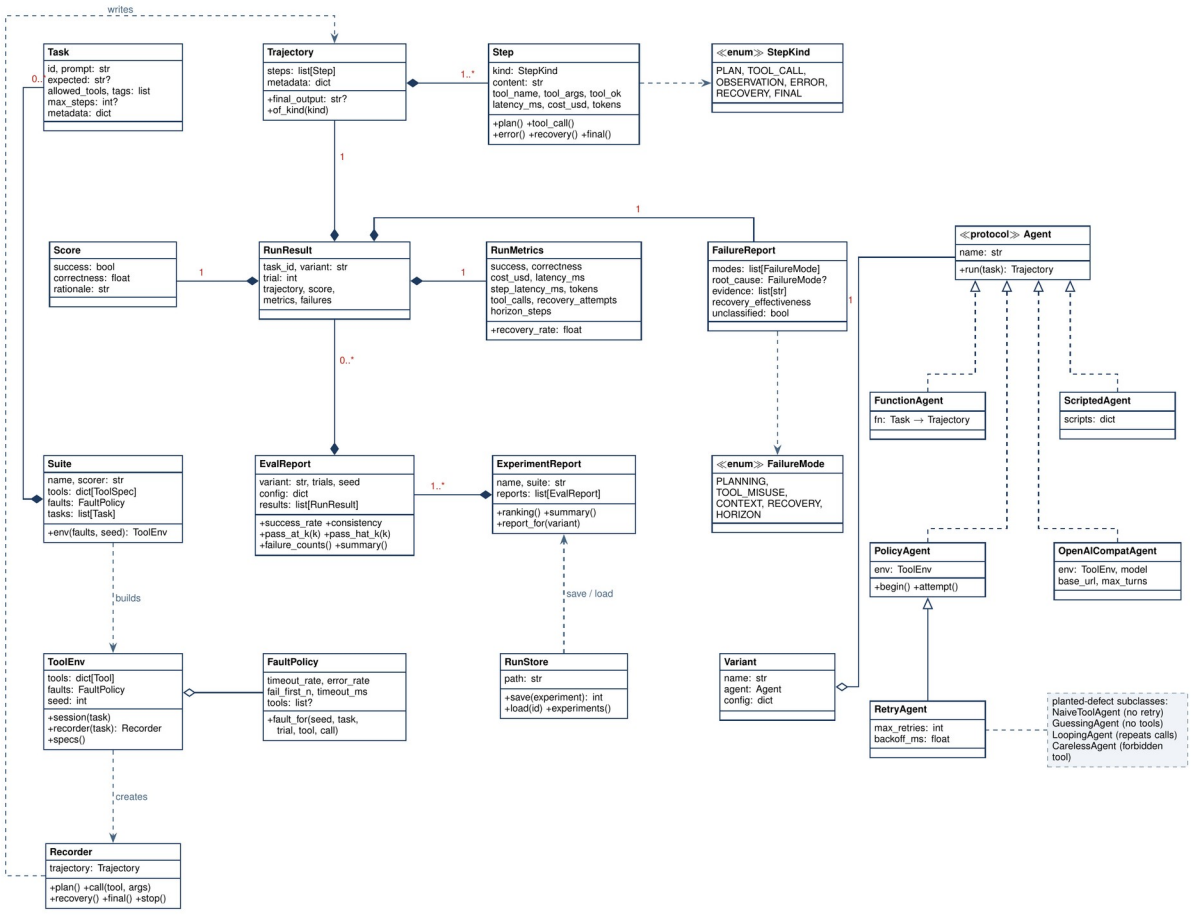


Figure 4.7: Class diagram

Sequence (Fig. 4.8). The command line loads the suite and environment, the harness loops over variants, tasks, and trials, the tool session decides each fault from the seed, and the analysis layer returns the score, metrics, and failure report.

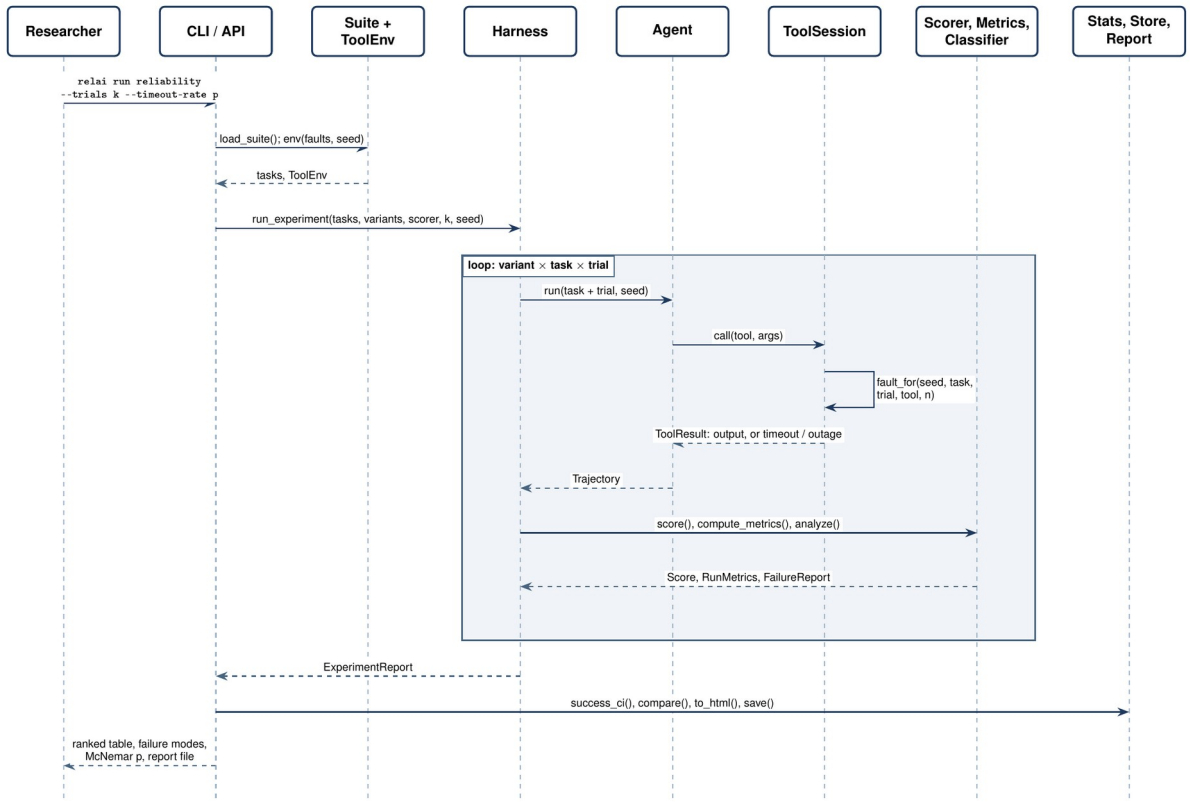


Figure 4.8: Sequence diagram of relai run

Data flow (Figs. 4.9 and 4.10). The context diagram places the researcher, the agent under test, and an optional model API around the laboratory; level 1 divides it into load, execute, score, classify, compare, and store processes with four data stores.

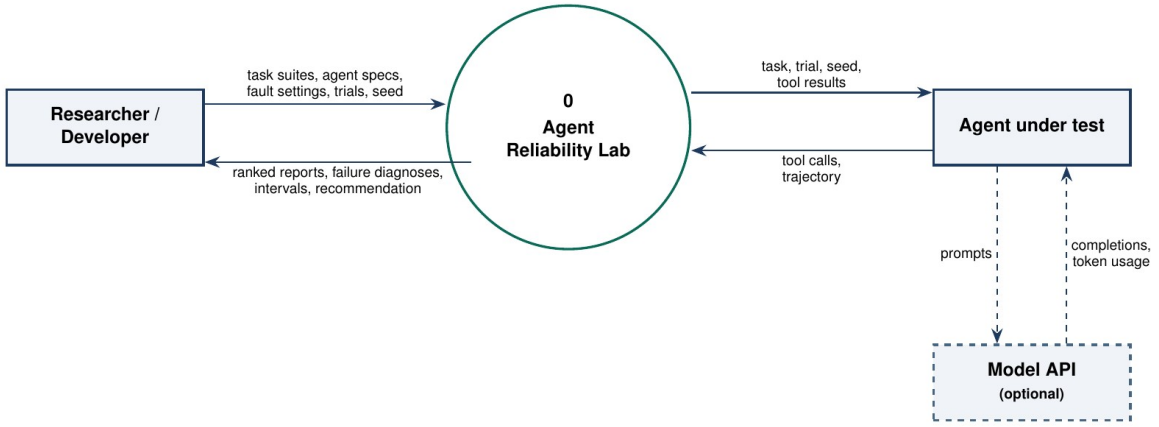


Figure 4.9: Level 0 data flow diagram

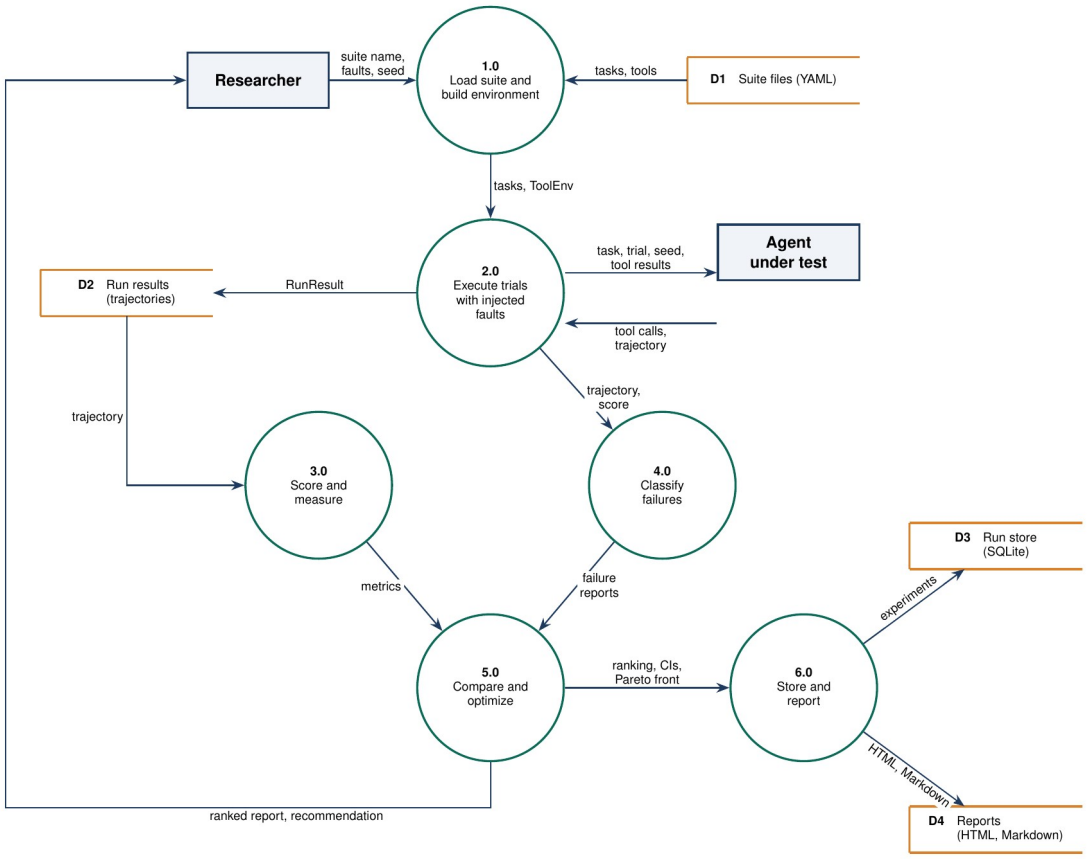


Figure 4.10: Level 1 data flow diagram

Flowcharts (Figs. 4.11 to 4.13). The evaluation loop, the failure classifier, and the configuration search are shown below.

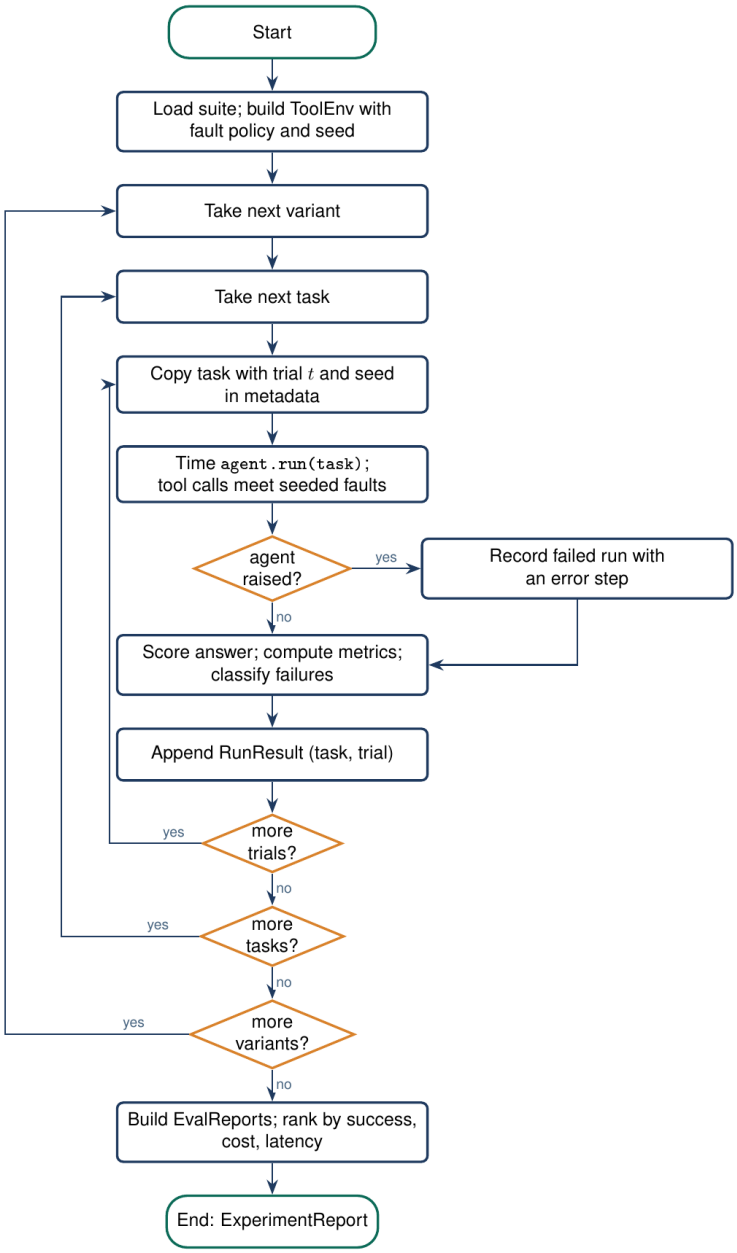


Figure 4.11: Flowchart of the evaluation harness

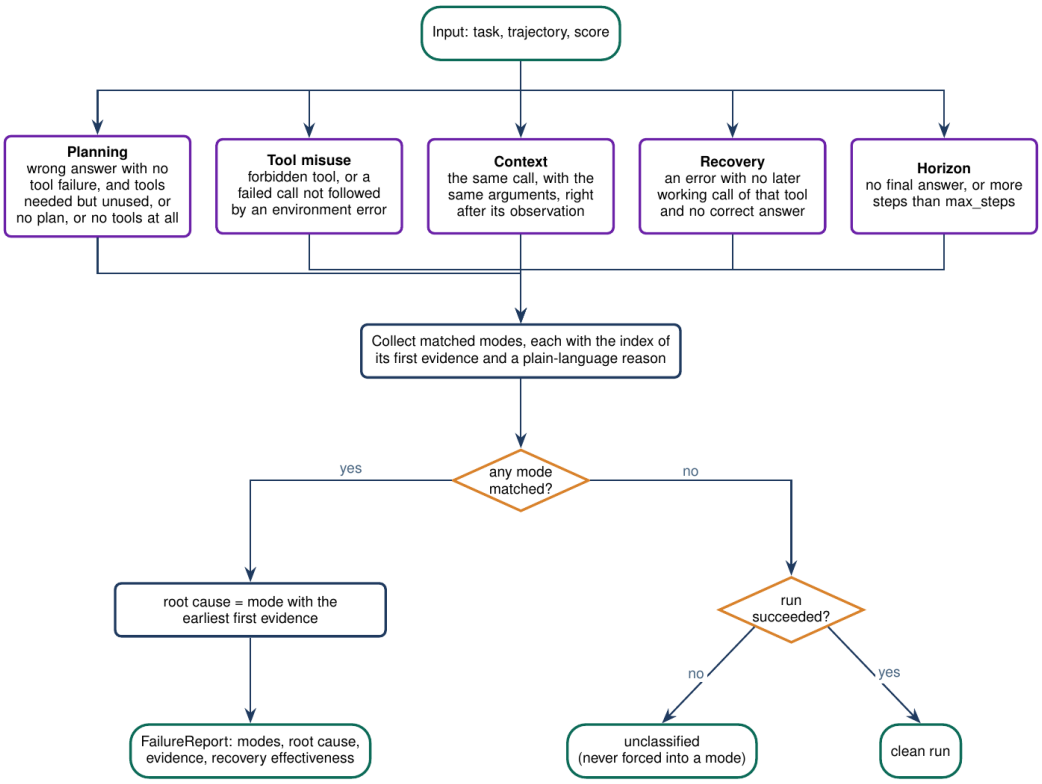


Figure 4.12: Flowchart of failure classification

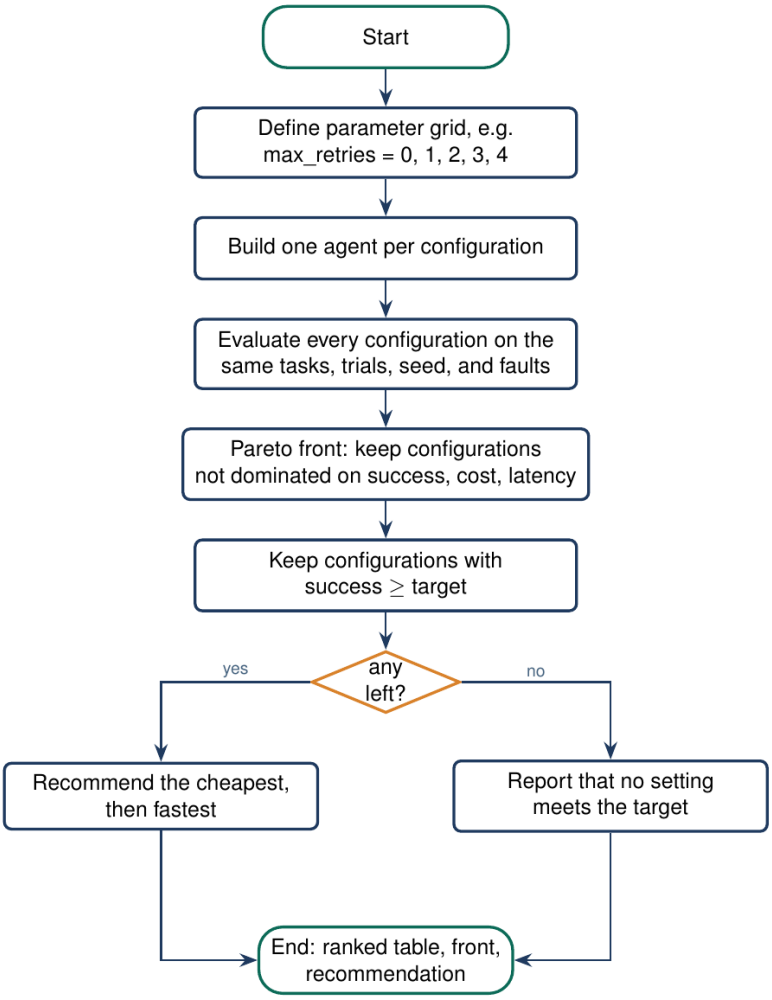


Figure 4.13: Flowchart of configuration search

4.5 Implementation Details

The implementation is a Python package of 17 modules and about 3,300 lines, with 81 tests. The main modules and their algorithms are described below; the code shown is taken directly from the repository with its real line numbers.

Tool environment and seeded fault injection (env.py)

Every tool call goes through a ToolSession created per run. For the n-th call to a tool, the session seeds a pseudo-random generator with the string (seed, task, trial, tool, n) and draws a number in [0, 1). The call times out if it is within the fail-first window or the draw is below the timeout rate, returns an outage if the draw falls in the next band, and otherwise runs the tool (Fig. 4.14). Because the draw ignores the agent, two agents that make the same calls meet the same faults. Unknown tools and invalid arguments fail without an error step, so they are recorded as misuse by the agent. The built-in calculator parses arithmetic with an abstract syntax tree and a whitelist of operators instead of eval().

src/relai/env.py (line 46)

```python
def fault_for(self, *, seed: int, task_id: str, trial: int, tool: str, call: int) -> str | None:
    """Return the fault for the call-th call of a tool in one run, if any."""
    if not self.applies_to(tool):
        return None
    if call <= self.fail_first_n:
        return "timeout"
    draw = random.Random(f"{seed}:{task_id}:{trial}:{tool}:{call}").random()
    if draw < self.timeout_rate:
        return "timeout"
    if draw < self.timeout_rate + self.error_rate:
        return "unavailable"
    return None
```

Figure 4.14: Fault decision for one tool call

Evaluation harness (harness.py)

The harness runs each task for k trials (Fig. 4.15). It passes the trial index and seed to the agent through the task metadata, so the agent interface needs no extra arguments, measures wall-clock time around the run, converts any exception into a failed run with an error step, and then scores, measures, and classifies the trajectory. With c successes in n trials of a task it estimates pass@k = 1 - C(n-c, k) / C(n, k) and pass^k = C(c, k) / C(n, k), averaged over tasks.

src/relai/harness.py (line 141)

```python
def evaluate(
    agent: Agent,
    tasks: Sequence[Task],
    scorer: Scorer = exact_match,
    *,
    trials: int = 1,
    seed: int = 0,
) -> EvalReport:
    """Run one agent on every task, trials times each, and return scored reports.

    The trial index and seed reach the agent through task.metadata, so tool
    environments can replay the same fault schedule for every variant.
    """
    if trials < 1:
        raise ValueError("trials must be at least 1")
    results: list[RunResult] = []
    for task in tasks:
        for trial in range(trials):
            run_task = task.model_copy(
                update={"metadata": {**task.metadata, TRIAL_KEY: trial, SEED_KEY: seed}},
                deep=True,
            )
            started = time.perf_counter()
            trajectory = _run_safely(agent, run_task)
            wall_ms = (time.perf_counter() - started) * 1000
            score = scorer(task, trajectory)
            metrics = compute_metrics(trajectory, score, wall_ms)
            failures = analyze(task, trajectory, score)
            results.append(
                RunResult(
                    task_id=task.id,
                    variant=agent.name,
                    trial=trial,
                    trajectory=trajectory,
                    score=score,
                    metrics=metrics,
                    failures=failures,
                )
            )
    return EvalReport(variant=agent.name, trials=trials, seed=seed, results=results)
```

Figure 4.15: Evaluation loop with trials and crash capture

Metrics and failure analysis (metrics.py, analysis.py)

Metrics are sums and counts over steps: cost, latency, tokens, tool calls, repeated calls after an observation, and steps. The recovery rate is the share of error steps after which the run restored a working path, defined as a later successful call of the tool that failed or a correct final answer (Fig. 4.16). Five detectors then look for planning, tool-misuse, context, recovery, and horizon failures; each returns the index of its first evidence and a reason. The root cause is the mode with the earliest evidence, and a failed run that matches nothing is unclassified.

src/relai/analysis.py (line 160)

```python
def error_episodes(trajectory: Trajectory, score: Score) -> list[tuple[int, bool]]:
    """Each error step's index and whether the run later restored a working path.

    A path is restored by a later successful call of the tool that failed, or of
    any tool when the error follows no tool call, or by a correct final answer.
    Failed retries in between do not break the episode, so a chain of retries
    that ends in success counts every one of its errors as recovered.
    """
    steps = trajectory.steps
    episodes: list[tuple[int, bool]] = []
    for idx, step in enumerate(steps):
        if step.kind != StepKind.ERROR:
            continue
        previous = steps[idx - 1] if idx else None
        failed_tool = (
            previous.tool_name
            if previous is not None and previous.kind == StepKind.TOOL_CALL
            else None
        )
        recovered = False
        for later in steps[idx + 1 :]:
            if (
                later.kind == StepKind.TOOL_CALL
                and later.tool_ok
                and (failed_tool is None or later.tool_name == failed_tool)
            ):
                recovered = True
                break
            if later.kind == StepKind.FINAL:
                recovered = score.success
                break
        episodes.append((idx, recovered))
    return episodes
```

Figure 4.16: Recovery episodes

Statistics and optimizer (stats.py, optimize.py)

With one trial per task the success interval is a Wilson score interval; with several trials the runs of a task are correlated, so a seeded bootstrap resamples whole tasks. Two variants are paired by task and trial, and the exact McNemar test uses the pairs that only one of them passed. The optimizer builds one variant per point of a parameter grid, keeps those not dominated on success, cost, and latency (Fig. 4.17), and recommends the cheapest, then fastest, variant that meets the success target.

src/relai/optimize.py (line 81)

```python
def pareto_front(
    reports: Sequence[EvalReport],
    objectives: Sequence[Objective] = DEFAULT_OBJECTIVES,
) -> list[EvalReport]:
    """Variants that no other variant beats on every objective at once."""
    front = [
        report
        for report in reports
        if not any(dominates(other, report, objectives) for other in reports if other is not report)
    ]
    return ExperimentReport(reports=front).ranking()
```

Figure 4.17: Pareto front

Task suites and reference agents (suite.py, baselines.py, selfcheck.py)

A suite declares tools as lookup tables or a calculator, with cost and latency per call, and each task carries a reference tool plan in which "{0}" refers to the output of the first call (Fig. 4.18). The built-in reliability suite models a customer-support desk with twelve tasks that need zero to five calls. The reference agents follow these plans but differ in one behaviour: the naive agent never retries, the retry agent retries environment faults with optional backoff (Fig. 4.19), the guessing agent never calls tools, the looping agent repeats calls, and the careless agent first calls a forbidden tool. The self-check runs them under shared faults and tabulates the diagnosed root causes.

src/relai/suites/reliability.yaml

```yaml
      - {tool: calculator, args: {expression: "{0} + {1}"}}
  - id: latest_order_status
    prompt: What is the status of the latest order placed by ravi@example.com?
    expected: delivered
    allowed_tools: [customer_lookup, order_history, order_status]
    tags: [requires_tools, long-horizon]
    max_steps: 12
    guess: shipped
    solution:
      - {tool: customer_lookup, args: {email: ravi@example.com}}
      - {tool: order_history, args: {customer: "{0}"}}
      - {tool: order_status, args: {order: "{1}"}}
```

Figure 4.18: A three-call task in the reliability suite

src/relai/baselines.py (line 91)

```python
def attempt(self, rec: Recorder, tool: str, args: dict[str, Any]) -> ToolResult:
    result = rec.call(tool, args)
    retries = 0
    while not result.ok and result.fault is not None and retries < self.max_retries:
        retries += 1
        rec.recovery(
            f"retry {retries}/{self.max_retries} after {result.fault}",
            latency_ms=self.backoff_ms * 2 ** (retries - 1),
        )
        result = rec.call(tool, args)
    return result
```

Figure 4.19: Retry policy of the reference agent

Store, reports, command line, and LLM adapter

The store writes an experiment in one transaction and reloads it from `result_json`. The reporter renders Markdown and a standalone HTML page and escapes all trajectory text. The

command-line interface parses agent specifications such as `retry:3` or `module:factory`, and prints status messages to standard error so that JSON output stays clean. The OpenAI-compatible adapter runs a tool-calling loop against a chat completions endpoint through the same tool environment and records tokens, cost, and latency per step; transport errors are recorded as error steps, and running out of turns produces a horizon failure.

CHAPTER 5
TESTING

5.1 Testing Methodology

Testing combined automated unit tests, integration tests, command-level system tests, and experimental validation of the instrument itself. All automated tests are written with pytest, one test file per module, and run on every push through GitHub Actions on Python 3.11, 3.12, and 3.13 together with Ruff lint.

1. **Unit testing.** Each function is tested in isolation with hand-built trajectory fixtures, for example a trajectory with two timeouts and a successful retry. Every failure rule has fixtures for when it must fire and when it must stay silent.
2. **Integration testing.** Modules are tested together: reference agents run through the tool environment, harness, scorers, and classifier on the built-in suites; experiments round-trip through SQLite and JSON Lines.
3. **System testing.** Every command-line command is executed with its options, and its terminal, JSON, HTML, Markdown, and database outputs are checked.
4. **Adapter testing.** The LLM adapter is tested with a fake transport that returns scripted model responses, so the tests never need a network or an API key.
5. **Instrument validation.** Experiments check the system against known answers: measured success against the closed-form effect of faults, and diagnosed root causes against planted defects (Section 5.3).

During development the self-check exposed a real defect: the first recovery rule judged each error only up to the next error, so a chain of retries that ended in success was reported as a recovery failure, and the control agent showed thirteen false recovery failures. The rule was corrected, pinned with new fixtures, and the precision of every mode rose to 1.00.

5.2 Test Cases and Results

Table 5.1: Representative test cases

ID	Test case	Expected result	Result
TC1	Fault schedule repeats for the same seed and is identical across agents	Same faults every time	Pass
TC2	Schedule changes with trial and with seed	Different faults	Pass
TC3	Unknown tool and wrong argument name	Recorded as tool misuse, not environment error	Pass
TC4	Calculator given code or a huge exponent	Rejected with an error	Pass
TC5	Agent passing 2 of 4 trials on a task	pass^2 = 1/6, pass@2 = 5/6	Pass
TC6	Agent raises an exception	Failed run with error step; experiment continues	Pass
TC7	Required tool never called, wrong answer	Planning failure	Pass
TC8	Timeout followed by giving up	Recovery failure, not tool misuse	Pass
TC9	Two timeouts, two retries, then success	No failure; recovery effectiveness 1.0	Pass
TC10	No final answer, or steps over budget	Horizon failure; silent within budget	Pass
TC11	Correct tools, wrong final answer	Unclassified	Pass

ID	Test case	Expected result	Result
TC12	Numeric answer 9000 for expected 9984	Fail with partial credit 0.90	Pass
TC13	Wilson interval for 8 of 10	[0.4902, 0.9433]	Pass
TC14	McNemar with 6 discordant pairs all one way	p = 2/64	Pass
TC15	Variant worse on every objective	Dropped from Pareto front	Pass
TC16	Each planted-defect agent under faults	Root cause equals planted defect	Pass
TC17	Save and reload an experiment (SQLite, JSONL)	Identical summaries	Pass
TC18	Agent output containing a script tag	Escaped in HTML report	Pass
TC19	LLM adapter with a fake model and a fail-first fault	Tool loop, tokens, cost, retry credited	Pass
TC20	Every CLI command; invalid agent spec	Correct output; clean error with exit code 2	Pass

All 81 automated tests pass in under one second and lint is clean (Fig. 5.1). Table 5.2 shows how the tests are distributed across modules.

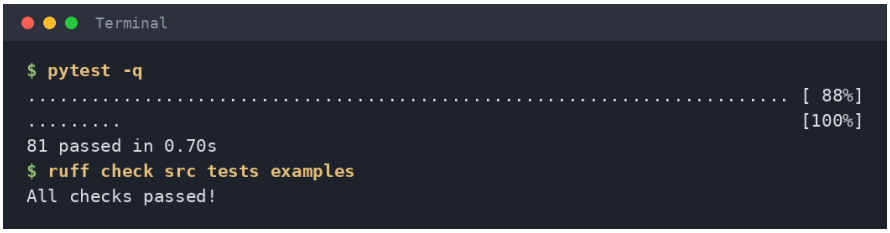


Figure 5.1: Test and lint output

Table 5.2: Distribution of automated tests

Test file	Tests	Covers
test_analysis.py	13	Every failure mode, retry chains, unclassified
test_env.py	10	Fault reproducibility, misuse versus faults, safe calculator
test_cli.py	10	Every command and error handling
test_metrics.py	9	Scorers and metric derivation
test_baselines.py	8	Planted defects and self-check
test_harness.py, test_stats.py, test_optimize.py	15	Trials, estimators, intervals, McNemar, Pareto
test_suite.py, test_store.py, test_report.py	12	Suites, persistence, HTML escaping
test_adapter.py, test_experiment.py	4	LLM tool loop, ranking
Total	81	All pass

5.3 Experimental Evaluation

The experiments use offline reference agents, not LLMs, so that the ground truth of every run is known; they validate the instrument rather than rank models. Unless noted, the reliability suite of twelve tasks is run with five trials per task, seed 0, and a tool timeout rate of 0.3, giving 60 runs per agent.

Reliability under faults

Table 5.3: Agents under 30% tool timeouts

Agent	Success [95% CI]	pass^5	Cost (USD)	Latency (ms)	Diagnosis
naive	0.58 [0.38, 0.77]	0.25	0.0016	543	25 recovery
retry, 1	0.77 [0.67, 0.88]	0.33	0.0025	947	14 recovery, 2 horizon
retry, 3	1.00 [1.00, 1.00]	1.00	0.0030	1116	12 horizon

Single-attempt success hides most of the difference in dependability: one retry raises success from 0.58 to 0.77 but pass^5 only from 0.25 to 0.33, while three retries reach 1.00 on both at about twice the cost and latency. The paired test finds 14 runs passed only by three retries and none passed only by one retry (exact McNemar p = 0.0001). Twelve runs of the three-retry agent answered correctly but exceeded their step budget and were flagged as horizon failures, which is the hidden cost of retrying. Fig. 5.2 shows how success and pass^5 fall as the timeout rate rises.

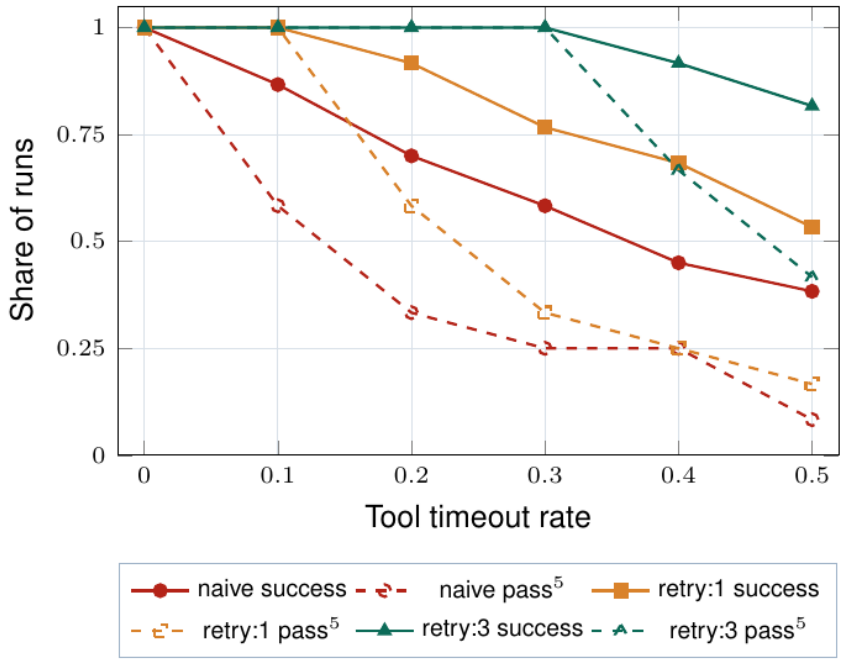


Figure 5.2: Success (solid) and pass^5 (dashed) as timeouts rise

Validation of the fault injector

For an agent that retries each failed call up to r times, a task that needs L calls succeeds with probability (1 - p^(r+1))^L when faults are independent with rate p. Table 5.4 and Fig. 5.3 compare this expectation with measurements over 200 trials per task; the close agreement shows that faults occur at the configured rate and independently across calls.

Table 5.4: Measured success against the closed-form expectation

Retries	Calls L	Runs	Measured	Expected
0	1	800	0.69	0.70
0	2	400	0.52	0.49

Retries	Calls L	Runs	Measured	Expected
0	3	800	0.33	0.34
0	5	200	0.16	0.17
1	1	800	0.90	0.91
1	2	400	0.82	0.83
1	3	800	0.76	0.75
1	5	200	0.60	0.62

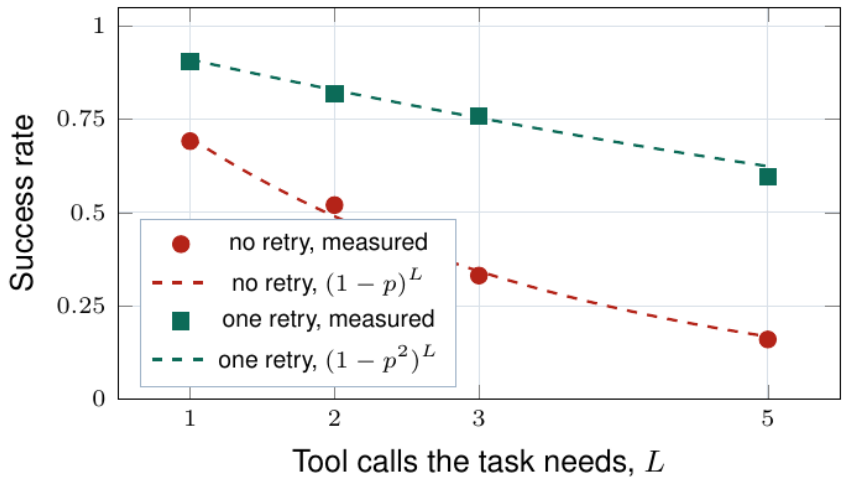


Figure 5.3: Measured success against the closed form

Classifier self-check

Table 5.5: Diagnosed root causes per planted defect (60 runs each)

Agent	Planted	Plan.	Misuse	Context	Recov.	Horizon	None
naive	recovery	0	0	0	25	0	35
guessing	planning	55	0	0	0	0	5
looping	context	0	0	55	0	0	5
careless	tool misuse	0	60	0	0	0	0
retry	control	0	0	0	1	11	48

Every diagnosed run of a planted agent is attributed to its planted mode, a precision of 1.00 for all four modes. Runs counted as none are runs where the defect never surfaced, such as a naive agent that met no timeout. The control agent shows one genuine recovery failure (three consecutive timeouts on one call) and eleven step-budget overruns.

Choosing a retry budget

Table 5.6: Sweep of the retry budget at 30% timeouts

Max retries	Success	pass^5	Cost (USD)	Latency (ms)	Horizon flags
0	0.58	0.25	0.0016	543	0
1	0.77	0.33	0.0025	947	2
2	0.98	0.92	0.0030	1116	12
3	1.00	1.00	0.0030	1116	12

Max retries	Success	pass^5	Cost (USD)	Latency (ms)	Horizon flags
4	1.00	1.00	0.0030	1116	12

Every setting lies on the Pareto front because each extra retry buys success with cost and latency. For a target of 0.95 the system recommends two retries, the cheapest setting that meets it.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project designed and implemented Agent Reliability Lab, a laboratory that turns the question of whether an AI agent works into controlled experiments on how consistently it works, why it fails, and what reliability costs. Its central idea is that tool failures should be part of the experiment, not part of the luck of a run: a seeded fault schedule that depends only on the task, trial, tool, and call number makes every variant meet the same failures, which turns comparisons into paired experiments with exact significance tests.

On top of this design the system provides repeated-trial reliability metrics, an evidence-based failure taxonomy that refuses to label what it cannot explain, confidence intervals, a cost-aware configuration search, persistence, reports, and a command-line interface, all in a small library that any agent can plug into. Testing with 81 automated tests and with experiments whose answers are known confirmed that the fault injector behaves as specified and that the classifier recovers planted defects.

The project strengthened skills in software architecture, test-first development, statistics for experiments, and honest evaluation: the results reported here validate the instrument, and the evaluation of live LLM agents is the next step.

6.2 Future Work

1. **Live LLM study.** Evaluate hosted and local models through the included adapter on the reliability suite, with real cost per model.
2. **Benchmark-derived suites.** Adapt tasks from public agent benchmarks into the suite format.
3. **More fault kinds.** Rate limits, slow responses, truncated or corrupted outputs, and changed tool schemas.
4. **Optional LLM judge.** A scorer for free-text answers and reasoning, kept separate from the rule-based classifier.
5. **Trace import.** Diagnose traces exported from tracing platforms such as Langfuse or OpenTelemetry.
6. **Dashboard.** A web view over the run store for browsing experiments and runs.
7. **Multi-agent systems.** Extend the taxonomy and map it onto published multi-agent failure taxonomies.
8. **Predictability and safety metrics.** Calibration of agent confidence and bounded error severity.
9. **Efficient experiments.** Parallel execution, adaptive search, and sequential tests that stop trials early to save API cost.
10. **Release.** Publish the research paper and release version 1.0 on the Python Package Index.

APPENDIX

A. Source Code

The complete source code, tests, task suites, design diagrams, and the scripts that regenerate every result in this report are in the project repository: https://github.com/devSuryansh/relAI

B. Running the Project

Install with `python -m venv .venv`, activate it, and run `pip install -e ".[dev]"`. Then `pytest -q` runs the tests and `relai run reliability --agent naive --agent retry:3 --trials 5 --timeout-rate 0.3` reproduces Table 5.3. All result files are regenerated with `python academic/results/make_results.py`.

C. Division of Work

Member	Responsibility
Suryansh Singh (team lead)	Architecture, domain types, agent interface, tool environment and fault injection, harness, experiments, store, CLI, LLM adapter, integration
Simran Kumari	Metrics and scorers, failure taxonomy and analysis, statistics, optimizer, results analysis, literature survey
Tanishka Sahu	Task suites, reference agents, self-check, reports, tests and examples, diagrams, documentation

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models", ICLR, 2023.

[2] X. Liu et al., "AgentBench: Evaluating LLMs as Agents", ICLR, 2024.

[3] G. Mialon et al., "GAIA: A Benchmark for General AI Assistants", ICLR, 2024.

[4] C. E. Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?", ICLR, 2024.

[5] S. Zhou et al., "WebArena: A Realistic Web Environment for Building Autonomous Agents", ICLR, 2024.

[6] S. Yao, N. Shinn, P. Razavi, K. Narasimhan, "tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains", arXiv:2406.12045, 2024.

[7] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, A. Narayanan, "AI Agents That Matter", arXiv:2407.01502, 2024.

[8] S. Kapoor et al., "Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation", ICLR, 2026.

[9] S. Rabanser, S. Kapoor, P. Kirgis, K. Liu, S. Utpala, A. Narayanan, "Towards a Science of AI Agent Reliability", ICML, PMLR vol. 306, 2026.

[10] Y. Ruan et al., "Identifying the Risks of LM Agents with an LM-Emulated Sandbox", ICLR, 2024.

[11] "ToolMisuseBench: An Offline Deterministic Benchmark for Tool Misuse and Recovery in Agentic Systems", arXiv:2604.01508, 2026.

[12] "Beyond Function Calling: Benchmarking Tool-Using Agents under Tool-Environment Unreliability", arXiv:2606.25819, 2026.

[13] "AgentChaos: Chaos Engineering for Agent Systems via Programmatic Fault Injection", arXiv:2608.06790, 2026.

[14] M. Cemri et al., "Why Do Multi-Agent LLM Systems Fail?", arXiv:2503.13657, 2025.

[15] D. Deshpande, V. Gangal, H. Mehta, J. Krishnan, A. Kannappan, R. Qian, "TRAIL: Trace Reasoning and Agentic Issue Localization", arXiv:2505.08638, 2025.

[16] S. Zhang et al., "Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems", ICML, PMLR vol. 267, 2025.

[17] P. Liang et al., "Holistic Evaluation of Language Models", Transactions on Machine Learning Research, 2023.

[18] UK AI Security Institute, "Inspect: A Framework for Large Language Model Evaluations", 2024. Available: https://inspect.aisi.org.uk

[19] M. Chen et al., "Evaluating Large Language Models Trained on Code", arXiv:2107.03374, 2021.

[20] A. Basiri et al., "Chaos Engineering", IEEE Software, vol. 33, no. 3, pp. 35-41, 2016.

[21] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages", Psychometrika, vol. 12, no. 2, pp. 153-157, 1947.

[22] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference", Journal of the American Statistical Association, vol. 22, no. 158, pp. 209-212, 1927.

[23] B. Efron, "Bootstrap Methods: Another Look at the Jackknife", Annals of Statistics, vol. 7, no. 1, pp. 1-26, 1979.

[24] T. G. Dietterich, "Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms", Neural Computation, vol. 10, no. 7, pp. 1895-1923, 1998.

[25] Pydantic, "Pydantic Documentation", Available: https://docs.pydantic.dev

[26] Python Software Foundation, "sqlite3: DB-API 2.0 Interface for SQLite Databases", Available: https://docs.python.org/3/library/sqlite3.html